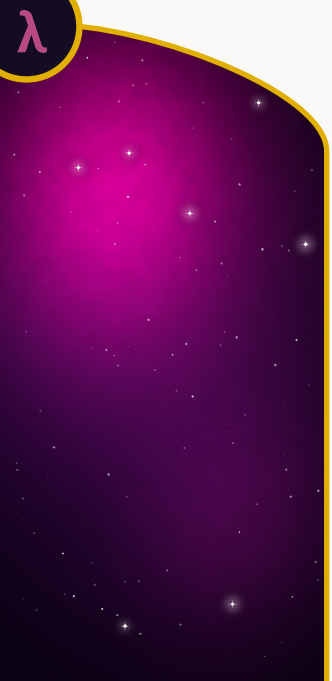


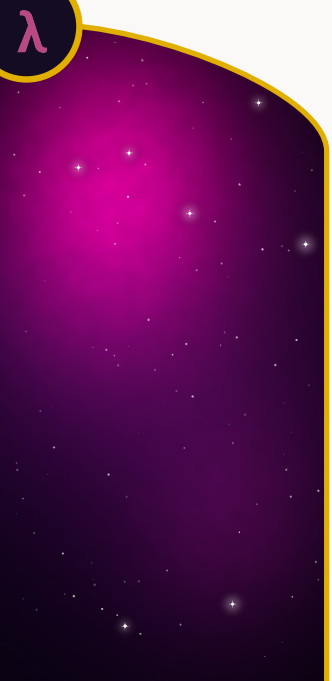
L4A: Introduction to Data Abstraction

CS1101S: Programming Methodology

Sanka Rasnayaka

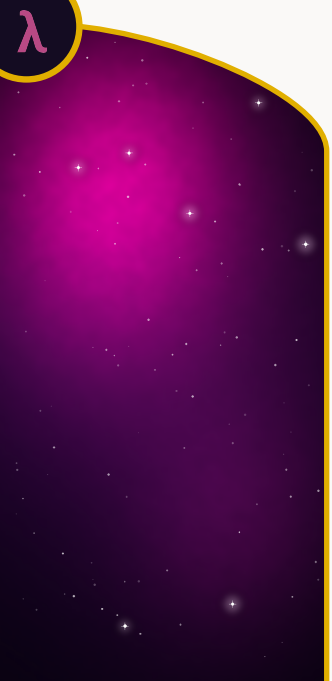
2nd September, 2026

Version 1.1 — Built: 2026-09-02 09:13:51 SGT



Outline

- Data abstraction (2.1)
  https://sourceacademy.nus.edu.sg/sicpy/2.1
- Case study: rational numbers (2.1.1)
  https://sourceacademy.nus.edu.sg/sicpy/2.1.1
- Representing sequences with pairs (2.2.1)
  https://sourceacademy.nus.edu.sg/sicpy/2.2.1



Announcements

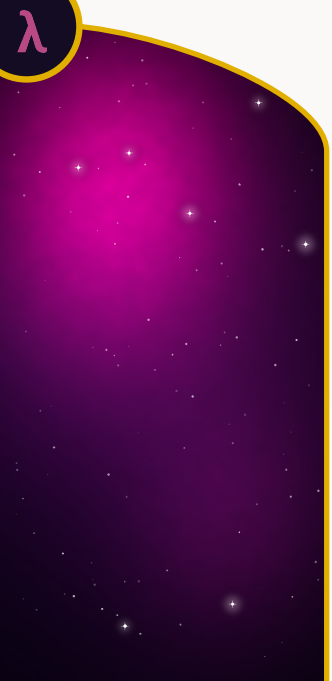
- RA1 this Friday!

Announcements

- RA1 this Friday!
- Contest "Beautifully Built" closes soon!

Announcements

- RA1 this Friday!
- Contest "Beautifully Built" closes soon!
- Unit 1 Survey open, closes Friday 23:59
  - In Google Form
  - Get 200 XP

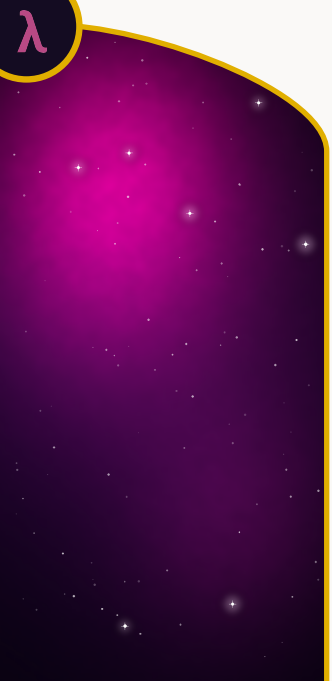


Where Are We Now?

- Module overview
  - Unit 1 — Functional abstraction: SICP Chapter 1
  - Unit 2 — Data abstraction: SICP Chapter 2
  - Unit 3 — State: SICP Chapter 3
  - Unit 4 — Beyond: SICP Chapters 4 & 5

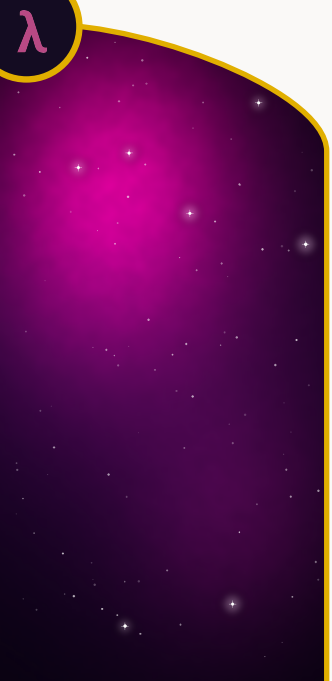
Where Are We Now?

- Module overview
  - Unit 1 — Functional abstraction: SICP Chapter 1
  - Unit 2 — Data abstraction: SICP Chapter 2
  - Unit 3 — State: SICP Chapter 3
  - Unit 4 — Beyond: SICP Chapters 4 & 5
- New language: Python §2
  - See language documentation at `https://docs.sourceacademy.org/python/python_2/`
  - Source Academy Playground is using Python §2 from now on `https://sourceacademy.nus.edu.sg/playground`



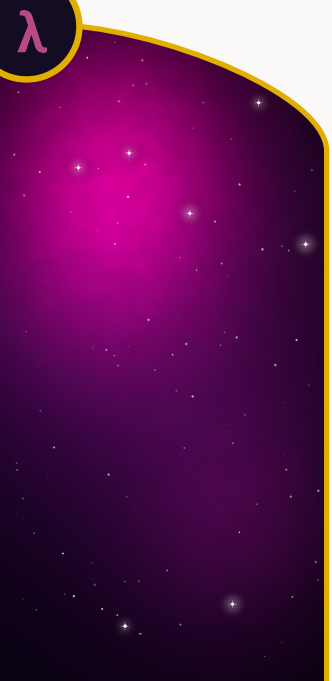
Outline

- Data abstraction (2.1)
  `https://sourceacademy.nus.edu.sg/sicpy/2.1`
- Case study: rational numbers (2.1.1)
- Representing sequences with pairs (2.2.1)



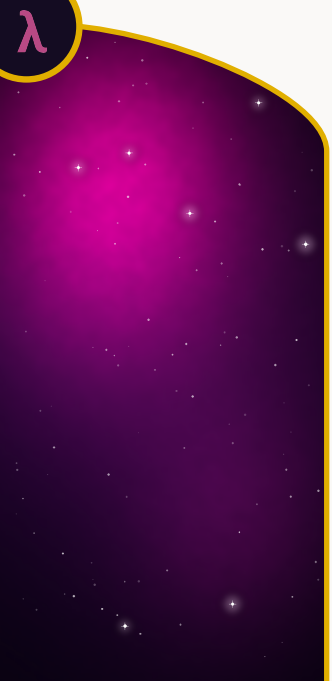
Types of Values in Python

- Numbers: `1`, `-5.6`, `0.5e-157`



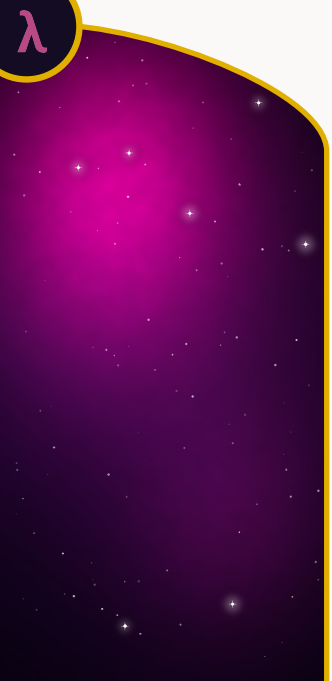
Types of Values in Python

- Numbers: `1`, `-5.6`, `0.5e-157`
- Boolean values: `True`, `False`



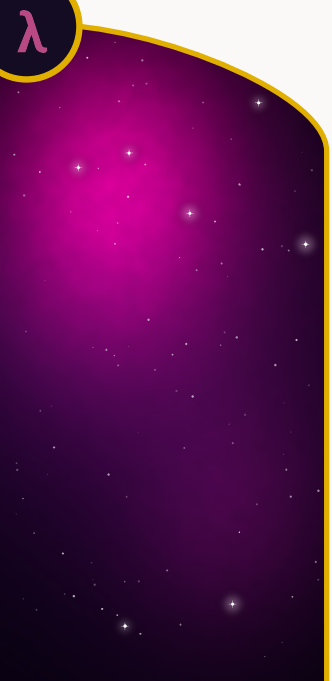
Types of Values in Python

- Numbers: `1`, `-5.6`, `0.5e-157`
- Boolean values: `True`, `False`
- Strings: `"this is a string"`



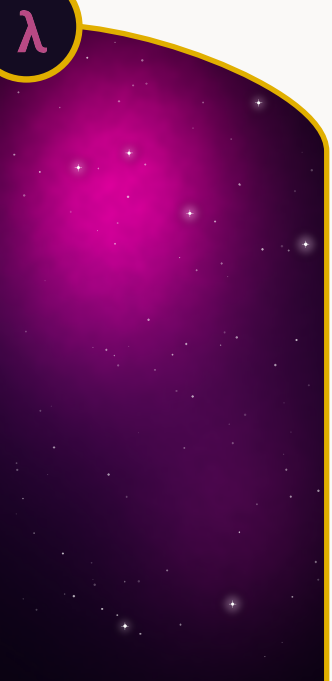
Types of Values in Python

- Numbers: `1, -5.6, 0.5e-157`
- Boolean values: `True, False`
- Strings: `"this is a string"`
- Functions: `lambda x: x + 1`



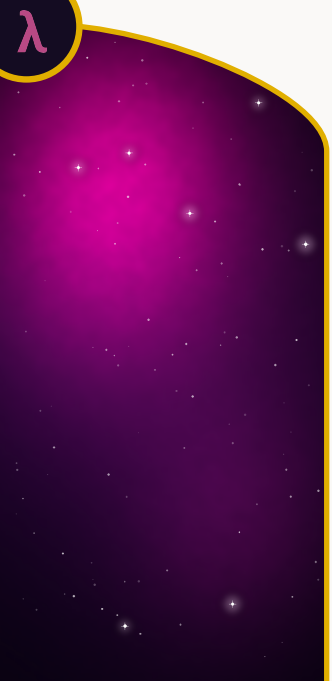
Types of Values in Python

- Numbers: `1`, `-5.6`, `0.5e-157`
- Boolean values: `True`, `False`
- Strings: `"this is a string"`
- Functions: `lambda x: x + 1`
- Some others (imported from modules): `heart`, `make_point(0.5, 0.25)`



Types of Values in Python

- Numbers: `1`, `-5.6`, `0.5e-157`
- Boolean values: `True`, `False`
- Strings: `"this is a string"`
- Functions: `lambda x: x + 1`
- Some others (imported from modules): `heart`, `make_point(0.5, 0.25)`
- Today: pairs and None



Recall `first_denomination` function in cc (L3)

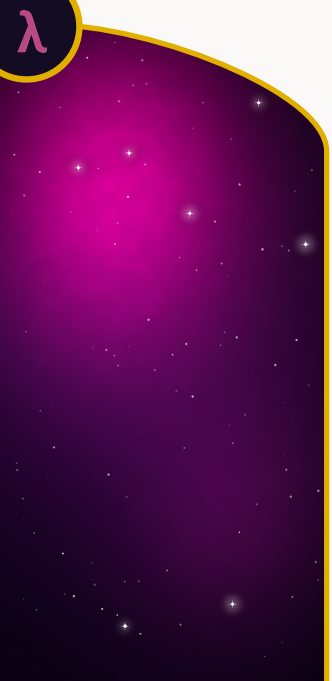
Recall `first_denomination` function in cc (L3)

```
def first_denomination(kinds_of_coins):
    return (5 if kinds_of_coins == 1
            else 10 if kinds_of_coins == 2
            else 20 if kinds_of_coins == 3
            else 50 if kinds_of_coins == 4
            else 100 if kinds_of_coins == 5
            else 0)
```

Recall `first_denomination` function in cc (L3)

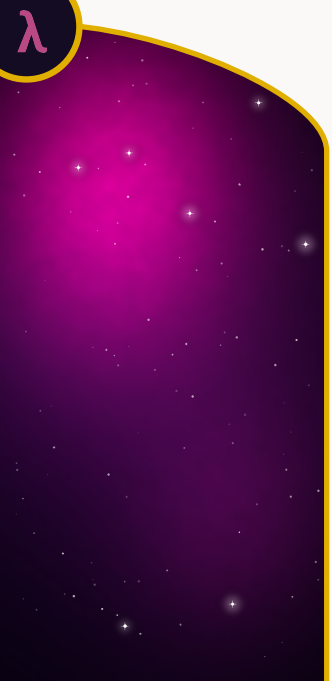
```
def first_denomination(kinds_of_coins):
    return (5 if kinds_of_coins == 1
            else 10 if kinds_of_coins == 2
            else 20 if kinds_of_coins == 3
            else 50 if kinds_of_coins == 4
            else 100 if kinds_of_coins == 5
            else 0)
```

- This `first_denomination` function "stores" five values and "retrieves" them when given the proper index



Data Structure Example

- `first_denomination` is a data structure



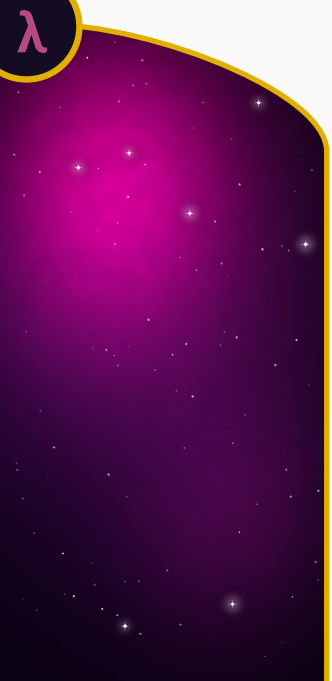
Data Structure Example

- `first_denomination` is a data structure
- Construct the data structure
  — using function definition

Data Structure Example

- `first_denomination` is a data structure
- Construct the data structure
  — using function definition

```
def first_denomination(kinds): ...
```



Data Structure Example

- `first_denomination` is a data structure
- Construct the data structure
  — using function definition

```
def first_denomination(kinds): ...
```

- Select components of the data structure
  — using function application

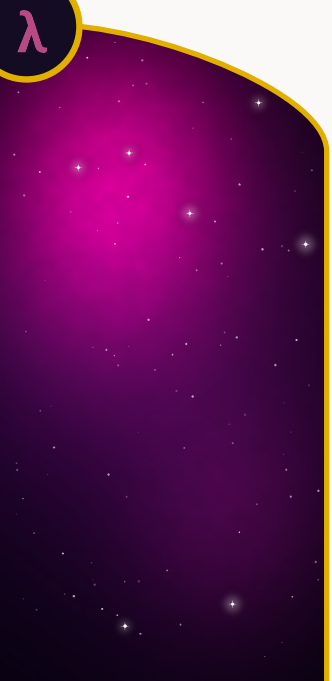
Data Structure Example

- `first_denomination` is a data structure
- Construct the data structure
  — using function definition

```
def first_denomination(kinds): ...
```

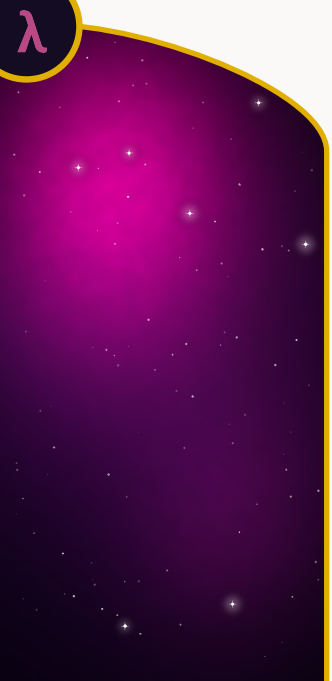
- Select components of the data structure
  — using function application

```
    ... first_denomination(3) ...
```



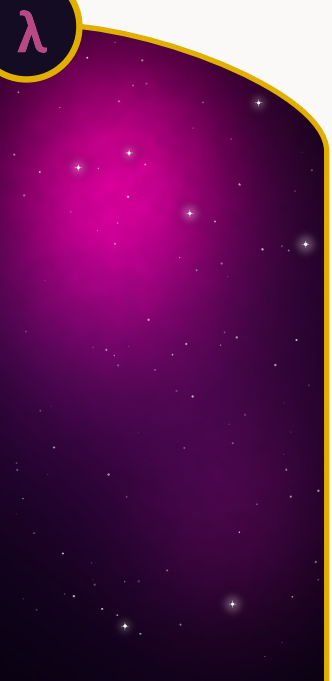
Data Structures in Mathematics

- They are **everywhere**: tuples, sets, matrices, etc.



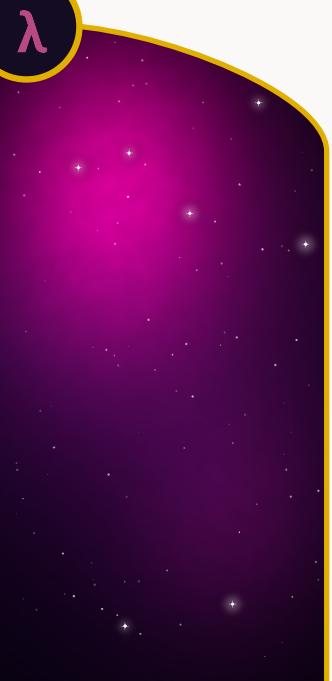
Data Structures in Mathematics

- They are **everywhere**: tuples, sets, matrices, etc.
- What is the simplest data structure possible?



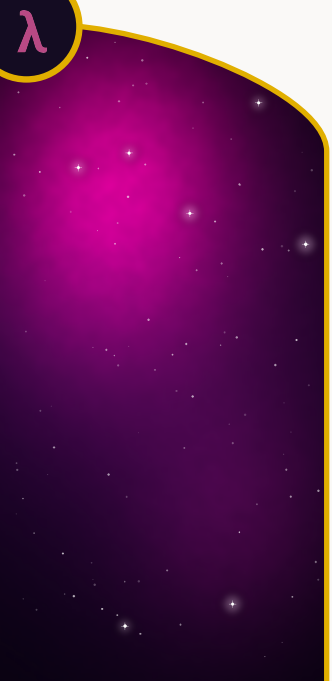
Data Structures in Mathematics

- They are **everywhere**: tuples, sets, matrices, etc.
- What is the simplest data structure possible?
- A pair



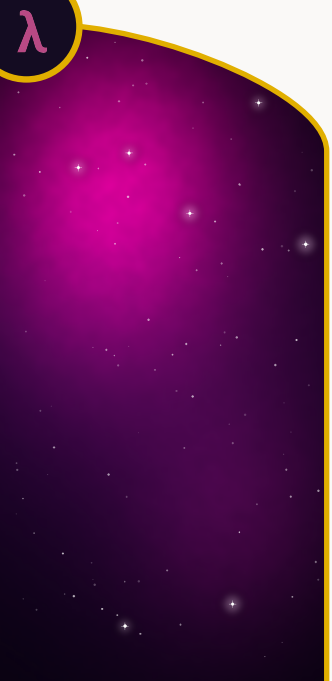
Data Structures in Mathematics

- They are **everywhere**: tuples, sets, matrices, etc.
- What is the simplest data structure possible?
- A pair
  - **Constructed** in math using tuple notation, e.g. (0.5, 0.25)



Data Structures in Mathematics

- They are **everywhere**: tuples, sets, matrices, etc.
- What is the simplest data structure possible?
- A pair
  - **Constructed** in math using tuple notation, e.g. (0.5, 0.25)
  - **Selected** in math using a pattern:
    - Let p be (x, y), for some x and y ... (and now use x and y)



Pairs so far — Points in Curve Missions

Pairs so far — Points in Curve Missions

```
# Construct:
p = make_point(0.5, 0.25)
```

Pairs so far — Points in Curve Missions

```
# Construct:
p = make_point(0.5, 0.25)

# Select:
x = x_of(p)
y = y_of(p)
```

We can Define the Point Abstraction (2.1.3)

```
def make_point(x, y):
    return lambda component: (
        x if component == 0 else y)
```

We can Define the Point Abstraction (2.1.3)

```
def make_point(x, y):
    return lambda component: (
        x if component == 0 else y)

def x_of(p):
    return p(0)
```

We can Define the Point Abstraction (2.1.3)

```
def make_point(x, y):
    return lambda component: (
        x if component == 0 else y)

def x_of(p):
    return p(0)

def y_of(p):
    return p(1)
```

Using More Generic Names

```
def pair(x, y):
    return lambda component: (
        x if component == 0 else y)

def head(p):
    return p(0)

def tail(p):
    return p(1)
```

Another way to define pair, head, tail

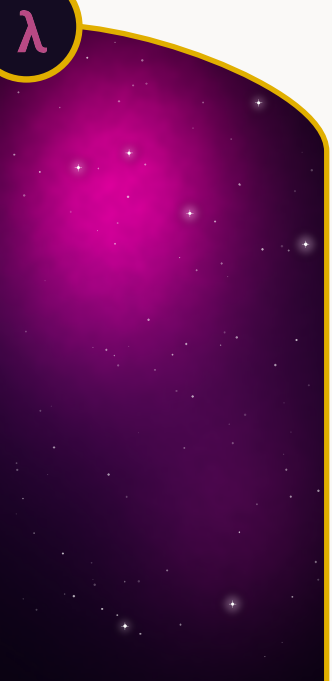
```
pair = lambda x, y: lambda f: f(x, y)
```

Another way to define pair, head, tail

```
pair = lambda x, y: lambda f: f(x, y)

head = lambda p: p(lambda x, y: x)
```

Another way to define pair, head, tail

```
pair = lambda x, y: lambda f: f(x, y)

head = lambda p: p(lambda x, y: x)

tail = lambda p: p(lambda x, y: y)
```

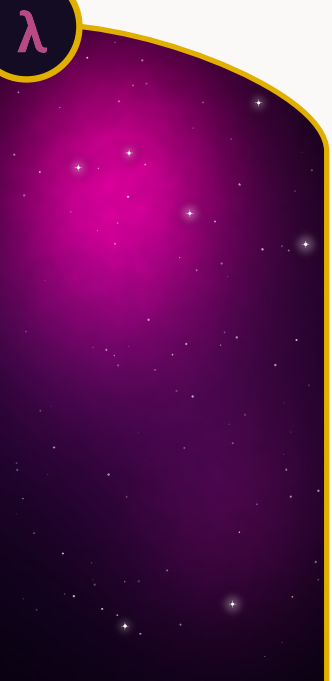


Outline

- Data abstraction (2.1)
- Case study: rational numbers (2.1.1)
  `https://sourceacademy.nus.edu.sg/sicpy/2.1.1`
- Representing sequences with pairs (2.2.1)

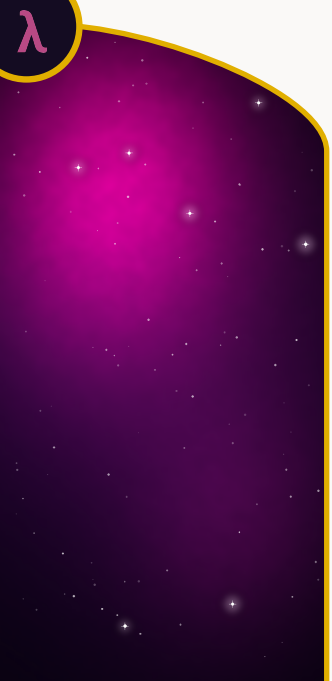
Case Study: Rational Numbers (2.1.1)

- What is a rational number?



Case Study: Rational Numbers (2.1.1)

- What is a rational number?
  - A pair, consisting of a denominator and a numerator



Case Study: Rational Numbers (2.1.1)

- What is a rational number?
  - A pair, consisting of a denominator and a numerator
- In Python:

Case Study: Rational Numbers (2.1.1)

- What is a rational number?
  - A pair, consisting of a denominator and a numerator
- In Python:

```
def make_rat(n, d):
    return pair(n, d)
```

Case Study: Rational Numbers (2.1.1)

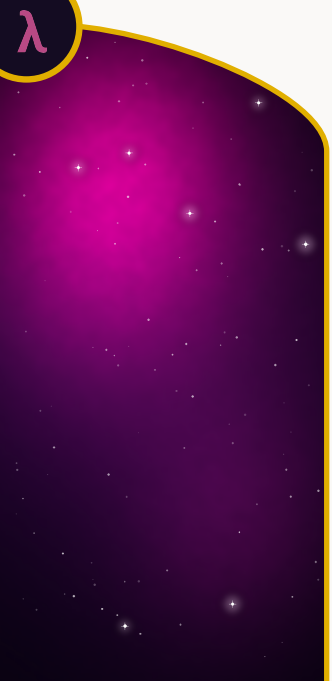
- What is a rational number?
  - A pair, consisting of a denominator and a numerator
- In Python:

```
def make_rat(n, d):
    return pair(n, d)

def numer(x):
    return head(x)
```

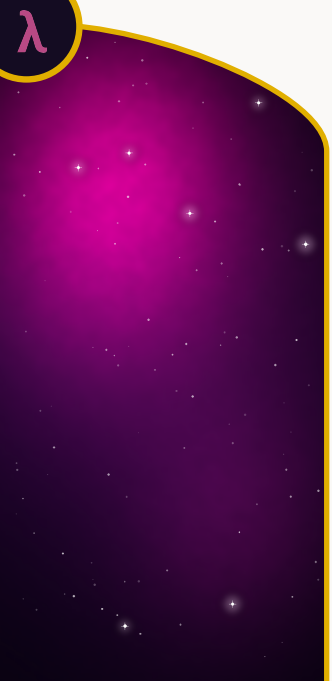
Case Study: Rational Numbers (2.1.1)

- What is a rational number?
  - A pair, consisting of a denominator and a numerator
- In Python:

```
def make_rat(n, d):
    return pair(n, d)

def numer(x):
    return head(x)

def denom(x):
    return tail(x)
```



Addition and Subtraction of Rational Numbers



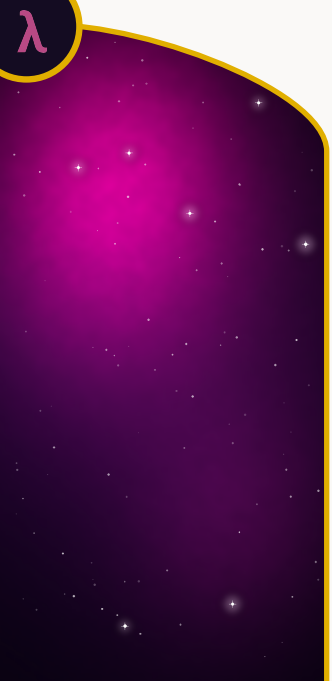
Addition and Subtraction of Rational Numbers

```
def add_rat(x, y):
    return make_rat(numer(x) * denom(y) +
                    numer(y) * denom(x),
                    denom(x) * denom(y))
```

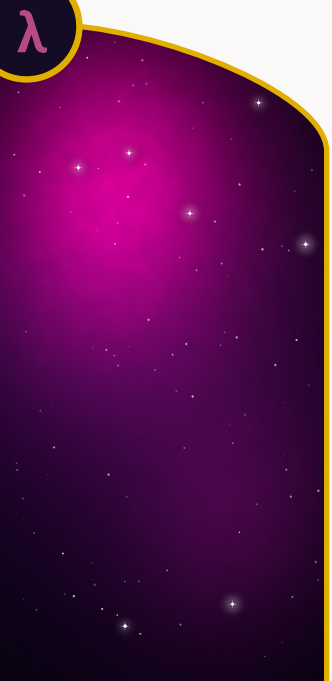
Addition and Subtraction of Rational Numbers

```
def add_rat(x, y):
    return make_rat(numer(x) * denom(y) +
                    numer(y) * denom(x),
                    denom(x) * denom(y))
```

```
def sub_rat(x, y):
    return make_rat(numer(x) * denom(y) -
                    numer(y) * denom(x),
                    denom(x) * denom(y))
```



Multiplication and Division of Rational Numbers



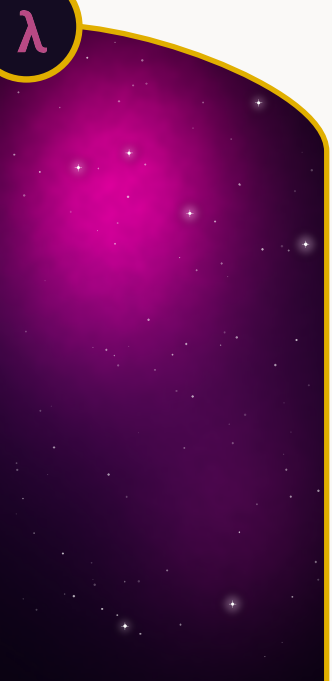
Multiplication and Division of Rational Numbers

```
def mul_rat(x, y):
    return make_rat(numer(x) * numer(y),
                    denom(x) * denom(y))
```

Multiplication and Division of Rational Numbers

```
def mul_rat(x, y):
    return make_rat(numer(x) * numer(y),
                    denom(x) * denom(y))
```

```
def div_rat(x, y):
    return make_rat(numer(x) * denom(y),
                    denom(x) * numer(y))
```



Equality of Rational Numbers

- First attempt:

Equality of Rational Numbers

- First attempt:

```
def equal_rat(x, y):
    return (numer(x) == numer(y) and
            denom(x) == denom(y))
```

Equality of Rational Numbers

- First attempt:

```
def equal_rat(x, y):
    return (numer(x) == numer(y) and
            denom(x) == denom(y))
```

- Second attempt:

Equality of Rational Numbers

- First attempt:

```
def equal_rat(x, y):
    return (numer(x) == numer(y) and
            denom(x) == denom(y))
```

- Second attempt:

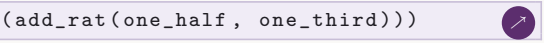
```
def equal_rat(x, y):
    return (numer(x) * denom(y) ==
            numer(y) * denom(x))
```

Printing Rational Numbers

```
def rat_to_string(x):
    return (str(numer(x)) + " / " +
        str(denom(x)))
```

Playing with Rational Numbers

```
print(rat_to_string(add_rat(one_half, one_third)))
```



- 5 / 6

Playing with Rational Numbers

```
print(rat_to_string(add_rat(one_half, one_third)))
```

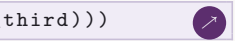
- `5 / 6`

```
print(rat_to_string(mul_rat(one_half, one_third)))
```

- `1 / 6`

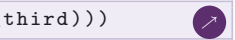
Playing with Rational Numbers

```
print(rat_to_string(add_rat(one_half, one_third)))
```



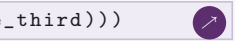
- 5 / 6

```
print(rat_to_string(mul_rat(one_half, one_third)))
```

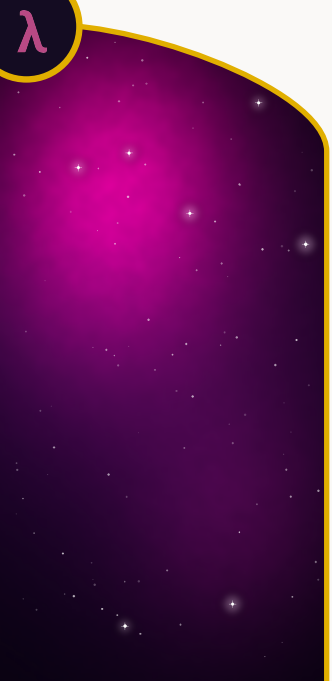


- 1 / 6

```
print(rat_to_string(add_rat(one_third, one_third)))
```



- 6 / 9



Making Reduced Rational Numbers

- Compute the greatest common divisor (GCD) of two numbers using Euclid's algorithm

Making Reduced Rational Numbers

- Compute the greatest common divisor (GCD) of two numbers using Euclid's algorithm

```
def gcd(a, b):
    return a if b == 0 else gcd(b, a % b)

def make_rat(n, d):
    g = gcd(n, d)
    return pair(n // g, d // g)
```

Playing with Rational Numbers Again

```
print(rat_to_string(add_rat(one_half, one_third)))
```

```
5 / 6
```

Playing with Rational Numbers Again

```
print(rat_to_string(add_rat(one_half, one_third)))
```

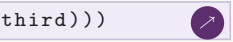
- `5 / 6`

```
print(rat_to_string(mul_rat(one_half, one_third)))
```

- `1 / 6`

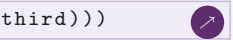
Playing with Rational Numbers Again

```
print(rat_to_string(add_rat(one_half, one_third)))
```



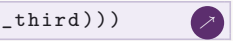
- `5 / 6`

```
print(rat_to_string(mul_rat(one_half, one_third)))
```



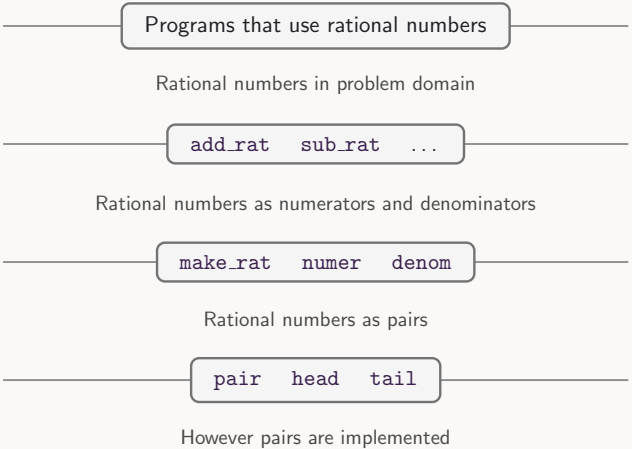
- `1 / 6`

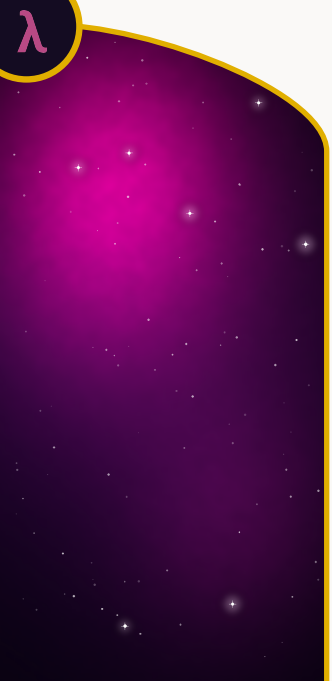
```
print(rat_to_string(add_rat(one_third, one_third)))
```



- `2 / 3`

Data Abstraction Barriers





Summary of Case Study on Rational Numbers

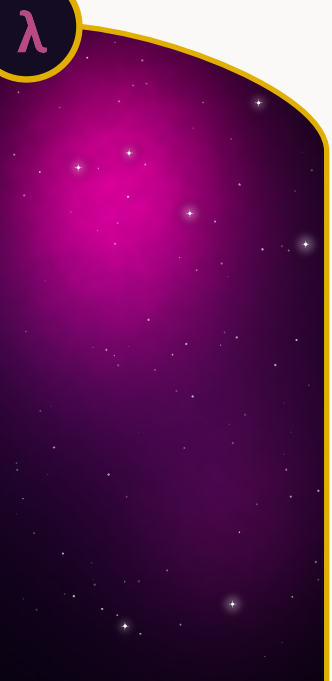
- Pairs can be used to represent rational numbers

Summary of Case Study on Rational Numbers

- Pairs can be used to represent rational numbers
- Operations are implemented using constructor and selector functions

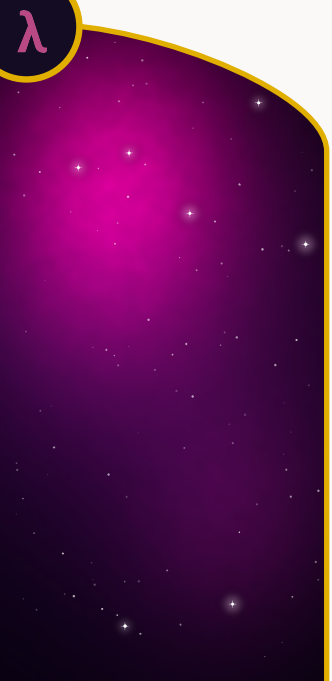
Summary of Case Study on Rational Numbers

- Pairs can be used to represent rational numbers
- Operations are implemented using constructor and selector functions
- `...rat...` functions hides the internal representation of the data
  - Implementation details remain invisible to the user of the library
  - Provides a higher-level abstraction



Outline

- Data abstraction (2.1)
- Case study: rational numbers (2.1.1)
- Representing sequences with pairs (2.2.1)
  `https://sourceacademy.nus.edu.sg/sicpy/2.2.1`



Representing Sequences with Pairs: Motivation

- Want to put the coin denominations 100, 50, 20, 10, 5 in a "sequence" constructed from pairs

Representing Sequences with Pairs: Motivation

- Want to put the coin denominations 100, 50, 20, 10, 5 in a "sequence" constructed from pairs

```
first_denomination = pair(...100...50...20...10...5...)
```

Representing Sequences with Pairs: Motivation

- Want to put the coin denominations 100, 50, 20, 10, 5 in a "sequence" constructed from pairs

```
first_denomination = pair(...100...50...20...10...5...)
```

- Many possible ways, examples:
  - `pair(pair(100, 50), pair(20, pair(10, 5)))`

Representing Sequences with Pairs: Motivation

- Want to put the coin denominations 100, 50, 20, 10, 5 in a "sequence" constructed from pairs

```
first_denomination = pair(...100...50...20...10...5...)
```

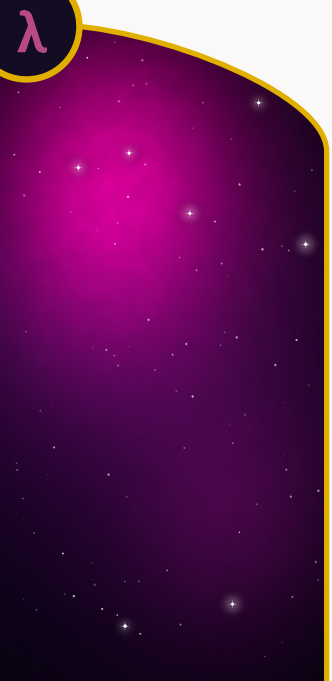
- Many possible ways, examples:
  - `pair(pair(100, 50), pair(20, pair(10, 5)))`
  - `pair(100, pair(pair(50, 20), pair(10, 5)))`
  - `pair(pair(pair(100, 50), pair(20, 10)), 5)`
  - `pair(100, pair(50, pair(20, pair(10, 5))))`

Representing Sequences with Pairs: Motivation

- Want to put the coin denominations 100, 50, 20, 10, 5 in a "sequence" constructed from pairs

```
first_denomination = pair(...100...50...20...10...5...)
```

- Many possible ways, examples:
  - `pair(pair(100, 50), pair(20, pair(10, 5)))`
  - `pair(100, pair(pair(50, 20), pair(10, 5)))`
  - `pair(pair(pair(100, 50), pair(20, 10)), 5)`
  - `pair(100, pair(50, pair(20, pair(10, 5))))`
- Different ways of representations require different ways of retrieval



Idea: Introduce Some Discipline

- Principle
  - Make sure that `head(p)` always has the **data**, and `tail(p)` always has the **remaining elements**

Idea: Introduce Some Discipline

- Principle
  - Make sure that `head(p)` always has the **data**, and `tail(p)` always has the **remaining elements**
- Example:

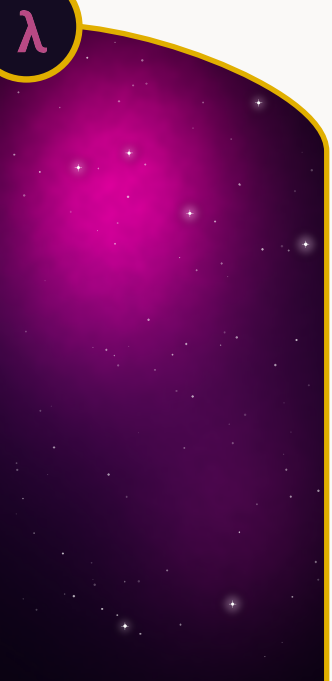
```
denoms = pair(100, pair(50, pair(20, pair(10, 5))))
```

Idea: Introduce Some Discipline

- Principle
  - Make sure that `head(p)` always has the **data**, and `tail(p)` always has the **remaining elements**
- Example:

```
denoms = pair(100, pair(50, pair(20, pair(10, 5))))
```

- `head(denoms)` $\Rightarrow$ `100`
- `tail(denoms)` $\Rightarrow$ `pair(50, pair(20, pair(10, 5)))`



Special Case

- What if

```
denoms = pair(10, 5)
```

Special Case

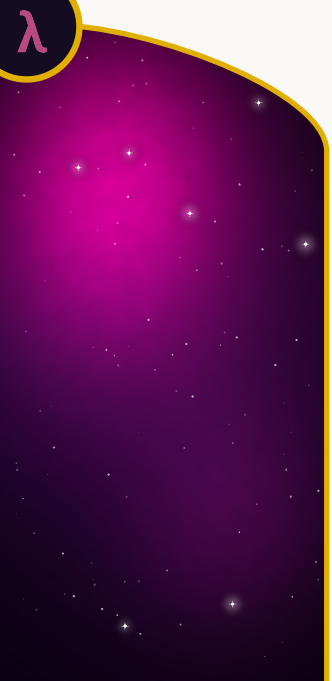
- What if

```
denoms = pair(10, 5)
```

- Then the program

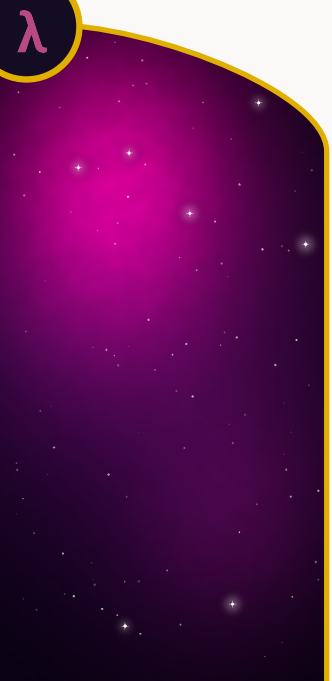
```
rest = tail(denoms)
```

gives us a value 5, not the remaining **sequence of elements**



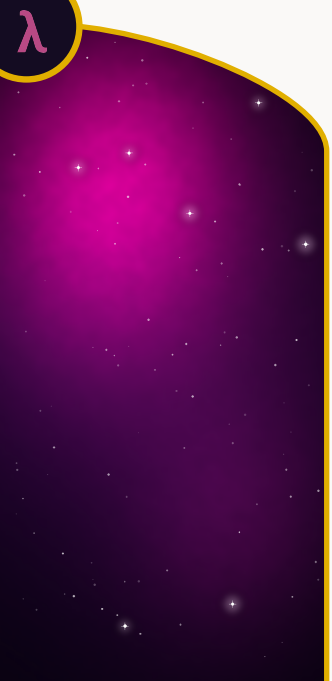
Idea: Introduce a Base Case

- How to represent the empty sequence?



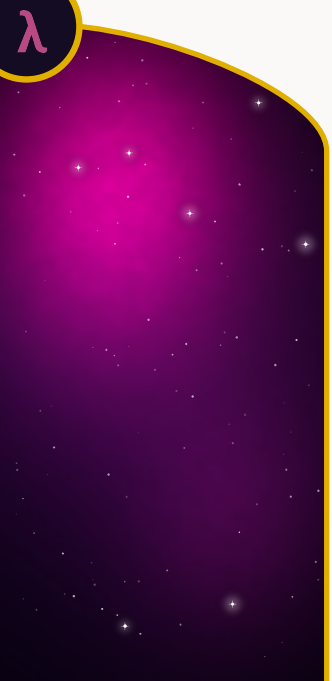
Idea: Introduce a Base Case

- How to represent the empty sequence?
  - It doesn't really matter!



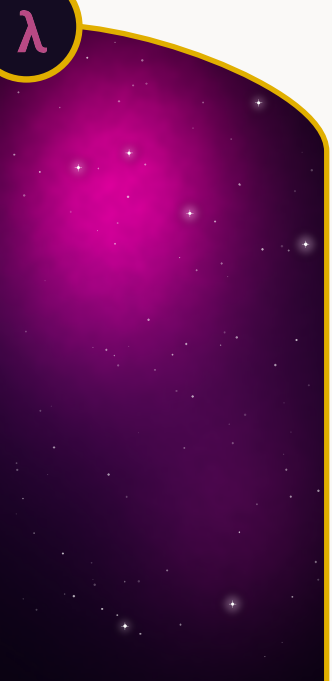
Idea: Introduce a Base Case

- How to represent the empty sequence?
  - It doesn't really matter!
- In Python, we use the special value `None` to represent a sequence of no elements



The first_denomination! using None

```
first_denomination = pair(100,
                          pair(50,
                               pair(20,
                                    pair(10,
                                         pair(5,
                                              None)))))
```

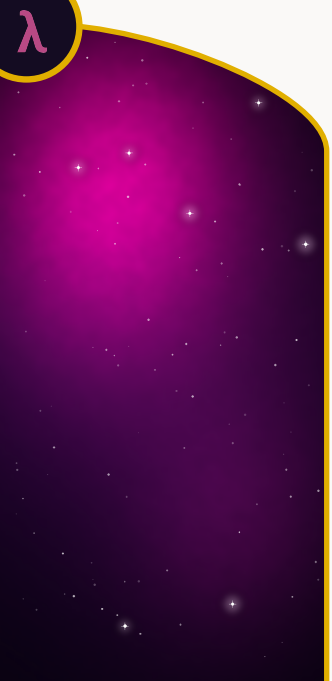


Linked List Discipline in Python

- Definition:

Linked List Discipline in Python

- Definition: A linked list is either `None` or a `pair` whose tail is a linked list



Linked List Discipline in Python

- Definition: A linked list is either `None` or a `pair` whose tail is a linked list
- We will call it a `llist` (li-list) for short

Linked List Discipline in Python

- Definition: A linked list is either `None` or a `pair` whose tail is a linked list
- We will call it a `llist` (li-list) for short

```
# Examples:
my_llist = None
```

Linked List Discipline in Python

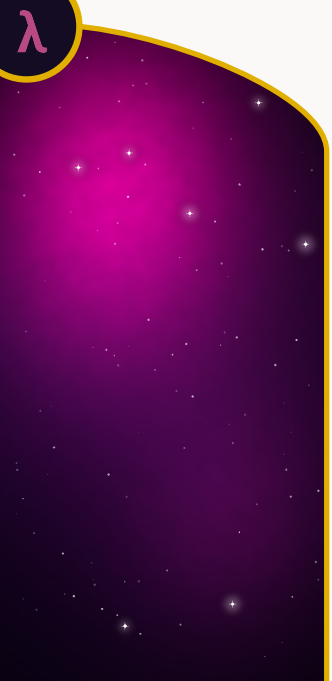
- Definition: A linked list is either `None` or a `pair` whose tail is a linked list
- We will call it a `llist` (li-list) for short

```
# Examples:
my_llist = None

your_llist = pair(8, None)
```

Linked List Discipline in Python

- Definition: A linked list is either `None` or a `pair` whose tail is a linked list
- We will call it a `llist` (li-list) for short

```
# Examples:
my_llist = None

your_llist = pair(8, None)

first_denomination =
    pair(100,
        pair(50,
            pair(20, pair(10, pair(5, None)))))
```



Retrieving Data from a Linked List

- Example:

Retrieving Data from a Linked List

- Example:

```
denoms =
    pair(100, pair(50, pair(20, pair(10, pair(5, None)))))
```

Retrieving Data from a Linked List

- Example:

```
denoms =
    pair(100, pair(50, pair(20, pair(10, pair(5, None)))))
```

`head(denoms)` $\Rightarrow$ `100`

Retrieving Data from a Linked List

- Example:

```
denoms =
    pair(100, pair(50, pair(20, pair(10, pair(5, None)))))
```

`head(denoms)` $\Rightarrow$ 100
`head(tail(denoms))` $\Rightarrow$ 50

Retrieving Data from a Linked List

- Example:

```
denoms =
    pair(100, pair(50, pair(20, pair(10, pair(5, None)))))
```

`head(denoms)` $\Rightarrow$ 100
`head(tail(denoms))` $\Rightarrow$ 50
`head(tail(tail(denoms)))` $\Rightarrow$ 20

Retrieving Data from a Linked List

- Example:

```
denoms =
    pair(100, pair(50, pair(20, pair(10, pair(5, None)))))
```

`head(denoms)` $\Rightarrow$ 100
`head(tail(denoms))` $\Rightarrow$ 50
`head(tail(tail(denoms)))` $\Rightarrow$ 20
`head(tail(tail(tail(denoms))))` $\Rightarrow$ 10

Retrieving Data from a Linked List

- Example:

```
denoms =
    pair(100, pair(50, pair(20, pair(10, pair(5, None)))))
```

head(denoms) $\Rightarrow$ 100
head(tail(denoms)) $\Rightarrow$ 50
head(tail(tail(denoms))) $\Rightarrow$ 20
head(tail(tail(tail(denoms)))) $\Rightarrow$ 10
head(tail(tail(tail(tail(denoms))))) $\Rightarrow$ 5

Retrieving Data from a Linked List

- Example:

```
denoms =
    pair(100, pair(50, pair(20, pair(10, pair(5, None)))))
```

head(denoms) ⇒ 100
head(tail(denoms)) ⇒ 50
head(tail(tail(denoms))) ⇒ 20
head(tail(tail(tail(denoms)))) ⇒ 10
head(tail(tail(tail(tail(denoms))))) ⇒ 5
tail(tail(tail(tail(tail(denoms))))) ⇒ None

LINKED LISTS Functions in (Python §2)

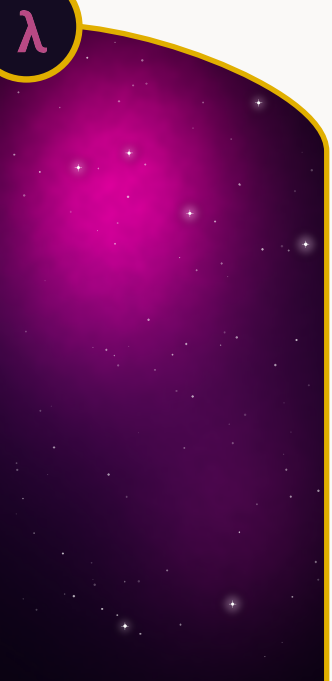
- `pair(x, y)` — returns pair made of x and y

LINKED LISTS Functions in (Python §2)

- `pair(x, y)` — returns pair made of x and y
- `is_pair(p)` — returns `True` iff p is a pair

LINKED LISTS Functions in (Python §2)

- `pair(x, y)` — returns pair made of x and y
- `is_pair(p)` — returns `True` iff p is a pair
- `None` — represents an empty linked list



LINKED LISTS Functions in (Python §2)

- `pair(x, y)` — returns pair made of x and y
- `is_pair(p)` — returns `True` iff p is a pair
- `None` — represents an empty linked list
- `is_none(xs)` — returns `True` iff xs is the empty linked list None

LINKED LISTS Functions in (Python §2)

- `pair(x, y)` — returns pair made of x and y
- `is_pair(p)` — returns `True` iff p is a pair
- `None` — represents an empty linked list
- `is_none(xs)` — returns `True` iff xs is the empty linked list None
- `head(p)` — returns the head (first component) of the pair p

LINKED LISTS Functions in (Python §2)

- `pair(x, y)` — returns pair made of x and y
- `is_pair(p)` — returns `True` iff p is a pair
- `None` — represents an empty linked list
- `is_none(xs)` — returns `True` iff xs is the empty linked list None
- `head(p)` — returns the head (first component) of the pair p
- `tail(p)` — returns the tail (second component) of the pair p

LINKED LISTS Functions in (Python §2)

- `pair(x, y)` — returns pair made of x and y
- `is_pair(p)` — returns `True` iff p is a pair
- `None` — represents an empty linked list
- `is_none(xs)` — returns `True` iff xs is the empty linked list None
- `head(p)` — returns the head (first component) of the pair p
- `tail(p)` — returns the tail (second component) of the pair p
- `llist(x1,...,xn)` — returns a linked list whose first element is x1, second element is x2, etc. and last element is xn

LINKED LISTS Functions in (Python §2)

- `pair(x, y)` — returns pair made of x and y
- `is_pair(p)` — returns `True` iff p is a pair
- `None` — represents an empty linked list
- `is_none(xs)` — returns `True` iff xs is the empty linked list None
- `head(p)` — returns the head (first component) of the pair p
- `tail(p)` — returns the tail (second component) of the pair p
- `llist(x1,...,xn)` — returns a linked list whose first element is x1, second element is x2, etc. and last element is xn
- ...

Box Notation

- In box notation

  `print(pair(x, y))` is printed as `[x, y]`
  Empty linked lists are printed as `None`

Box Notation

- In box notation

  `print(pair(x, y))` is printed as `[x, y]`
  Empty linked lists are printed as `None`
- Example:

```
print(pair(1, pair(2, pair(3, None))))
```

Box Notation

- In box notation

  `print(pair(x, y))` is printed as `[x, y]`
  Empty linked lists are printed as `None`
- Example:

```
print(pair(1, pair(2, pair(3, None))))
```

  is printed as

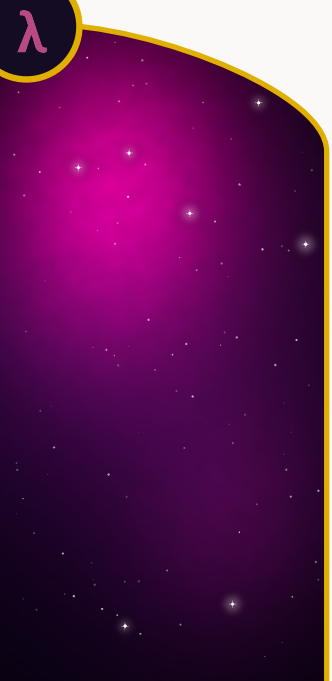
Box Notation

- In box notation

  `print(pair(x, y))` is printed as `[x, y]`
  Empty linked lists are printed as `None`
- Example:

```
print(pair(1, pair(2, pair(3, None))))
```

  is printed as
  `[1, [2, [3, None]]]`



Linked List Notation

- Same as box notation, but any sub-structure that is a llist is nicely formatted and printed as `llist(...)`
- Use predeclared function `print_llist(x)` to show x in llist notation

Linked List Notation

- Same as box notation, but any sub-structure that is a llist is nicely formatted and printed as `llist(...)`
- Use predeclared function `print_llist(x)` to show x in llist notation
- Example:

```
print_llist(
    pair(pair(pair(7, 8), pair(1, pair(2, None))), 6))
```

prints

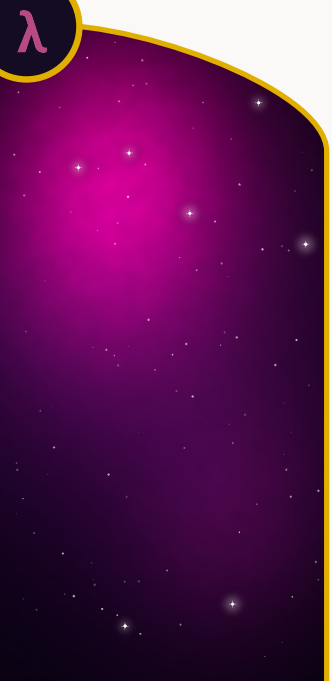
Linked List Notation

- Same as box notation, but any sub-structure that is a llist is nicely formatted and printed as `llist(...)`
- Use predeclared function `print_llist(x)` to show x in llist notation
- Example:

```
print_llist(
    pair(pair(pair(7, 8), pair(1, pair(2, None))), 6))
```

prints

```
[llist([7, 8], 1, 2), 6]
```



Box-and-Pointer Diagrams

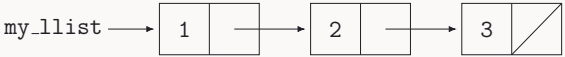
- Box-and-pointer diagrams are graphical representations of data structures made of pairs

Box-and-Pointer Diagrams

- Box-and-pointer diagrams are graphical representations of data structures made of pairs
  Example: `my_llist = pair(1, pair(2, pair(3, None)))`

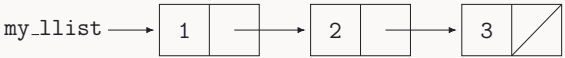
Box-and-Pointer Diagrams

- Box-and-pointer diagrams are graphical representations of data structures made of pairs
  Example: `my_llist = pair(1, pair(2, pair(3, None)))`



Box-and-Pointer Diagrams

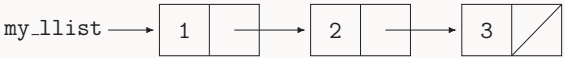
- Box-and-pointer diagrams are graphical representations of data structures made of pairs
  Example: `my_llist = pair(1, pair(2, pair(3, None)))`



- Data Visualizer tool generates such diagrams in Playground

Box-and-Pointer Diagrams

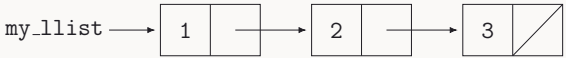
- Box-and-pointer diagrams are graphical representations of data structures made of pairs
  Example: `my_llist = pair(1, pair(2, pair(3, None)))`



- Data Visualizer tool generates such diagrams in Playground
- Use the `draw_data` primitive function

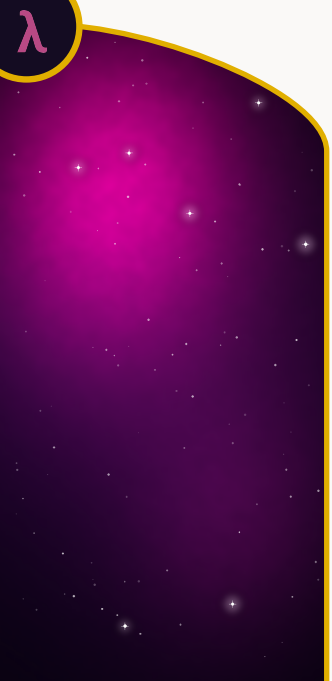
Box-and-Pointer Diagrams

- Box-and-pointer diagrams are graphical representations of data structures made of pairs
  Example: `my_llist = pair(1, pair(2, pair(3, None)))`

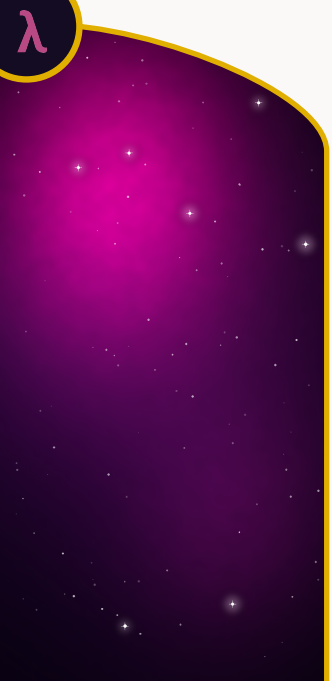


- Data Visualizer tool generates such diagrams in Playground
- Use the `draw_data` primitive function

```
draw_data(pair(1, pair(2, pair(3, None))))
```



Check-In I4A



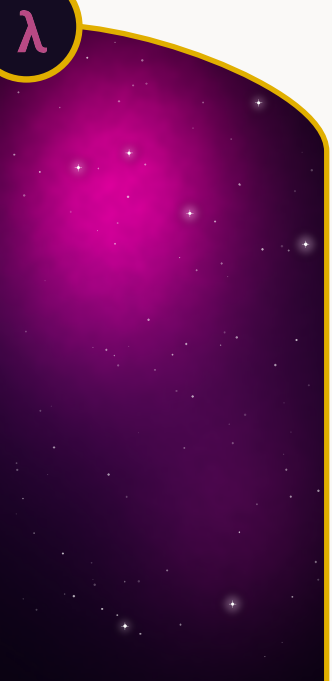
Check-In I4A

Time for a Sound Check(-In)!

Check-In I4A

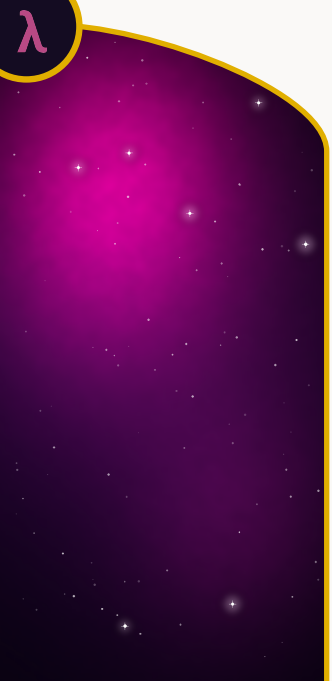
Time for a Sound Check(-In)!

Look for Check-In I4A in the Source Academy!



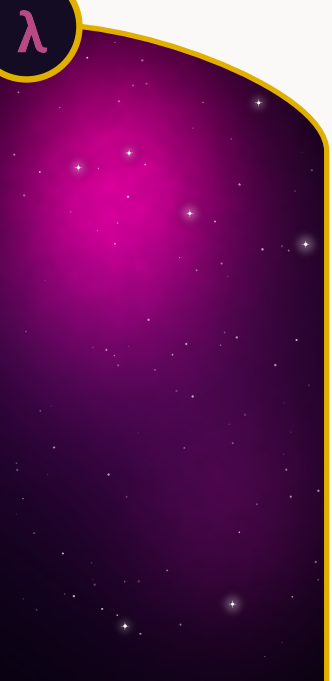
Error Reporting

- The functions that query the structure of linked lists have expectations for their arguments:
  - `head(xs)`: expects `xs` is a pair
  - `tail(xs)`: expects `xs` is a pair



Error Reporting

- The functions that query the structure of linked lists have expectations for their arguments:
  - `head(xs)`: expects `xs` is a pair
  - `tail(xs)`: expects `xs` is a pair
- Otherwise, a nice error message gets printed



Length of a Linked List

- Definition:

Length of a Linked List

- Definition:
  The length of the empty linked list is 0, and the length of a non-empty linked list is one more than the length of its tail

Length of a Linked List

- Definition:
  The length of the empty linked list is 0, and the length of a non-empty linked list is one more than the length of its tail
- Examples:
  - Length of `None` is 0

Length of a Linked List

- Definition:
  The length of the empty linked list is 0, and the length of a non-empty linked list is one more than the length of its tail
- Examples:
  - Length of `None` is 0
  - Length of `pair(10, None)` is 1

Length of a Linked List

- Definition:
  The length of the empty linked list is 0, and the length of a non-empty linked list is one more than the length of its tail
- Examples:
  - Length of `None` is 0
  - Length of `pair(10, None)` is 1
  - Length of `pair(10, pair(20, pair(30, None)))` is 3

Computing the Length of a Linked List

```
def length(xs):
    return (0
```

Computing the Length of a Linked List

```
def length(xs):
    return (0
            if is_none(xs)
```

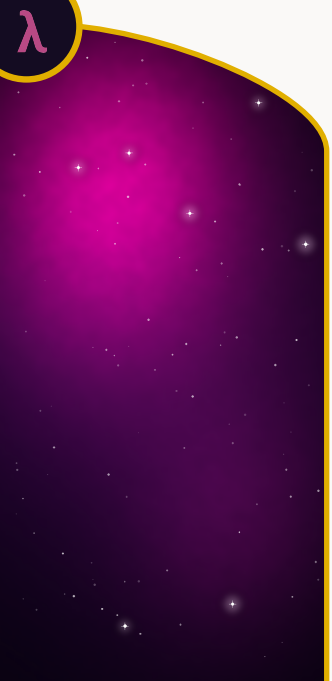
Computing the Length of a Linked List

```
def length(xs):
    return (0
            if is_none(xs)
            else 1 + length(tail(xs)))
```

Computing the Length of a Linked List

```
def length(xs):
    return (0
            if is_none(xs)
            else 1 + length(tail(xs)))
```

- Does it give rise to a recursive or iterative process?



Computing the Length of a Linked List

- Iterative version:

Computing the Length of a Linked List

- Iterative version:

```
def length_iter(xs):
```

Computing the Length of a Linked List

- Iterative version:

```
def length_iter(xs):
    def len(ys, total):
```

Computing the Length of a Linked List

- Iterative version:

```
def length_iter(xs):
    def len(ys, total):



    return len(xs, 0)
```

Computing the Length of a Linked List

- Iterative version:

```
def length_iter(xs):
    def len(ys, total):
        return (total


    return len(xs, 0)
```

Computing the Length of a Linked List

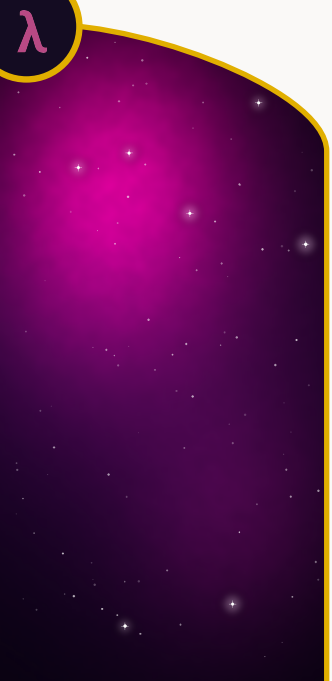
- Iterative version:

```
def length_iter(xs):
    def len(ys, total):
        return (total
                if is_none(ys)

    return len(xs, 0)
```

Computing the Length of a Linked List

- Iterative version:

```
def length_iter(xs):
    def len(ys, total):
        return (total
                if is_none(ys)
                else len(tail(ys), total+1))
    return len(xs, 0)
```



Summary

- Data structures are everywhere
- Pairs
- Case study: rational numbers
- Linked List discipline
- LINKED LISTS functions in Python §2
- Box notation, List notation, and Box-and-pointer diagrams